Same guardrail.
Three AI gateways.
5 of 5 attacks blocked.

13 of 24 real questions refused.

The same guardrail, installed inside three AI gateways.
The second number is the one nobody sells you.

SWIPE FOR THE NUMBERS

THE NUMBER

5 / 5

attacks blocked by LiteLLM and Portkey
with the policy on

0 / 24 flagged on the easy clean set:
guaranteed by construction, not measured

13 / 24 refused on the hard clean set, the
identical 13 on both gateways

2 of 3 ship the admin API reachable without
credentials in-network

HOW WE GOT HERE

MAR 2026

TeamPCP backdoors two LiteLLM releases on PyPI. Live for about forty minutes.

APR 2026

CVE-2026-42208: pre-auth SQL injection in LiteLLM Proxy. Exploited within 36 hours.

MAY 2026

Palo Alto Networks closes its acquisition of Portkey.

AUG 2026

This lab: three real gateways, pinned by digest, no egress, one policy installed in each.

WHY IT MATTERS

A gateway is an enforcement surface, not a detector. All three did exactly what they are built to do. The policy you install is where the security actually lives, and the policy that stopped every attack also refused more than half of a set of questions a real customer would plausibly ask.

Portkey refuses visibly. LiteLLM rewrites the question and answers anyway. The same 13 of 24. One failure you can see, one you cannot.

FIVE MOVES THIS QUARTER

1. Write a hard clean set before you trust any false-positive number, including your own.
2. Ask what your gateway does to a request it dislikes but does not refuse.
3. Close the admin API. “Internal network” is not an authentication scheme.
4. Pin gateway images by digest. A fixed tag is still a floating reference.
5. Budget for the proxy, not the guardrail. Scanning is nearly free; the hop is not.

Enforcement is not detection. Proving what a control actually does, and what it breaks, is the work. Full teardown and the reproducible lab in the comments.